GRADO EN INGENIERÍA INFORMÁTICA

ALGORÍTMICA Y MODELOS DE COMPUTACIÓN

MEMORIA DE LA PRÁCTICA 1

Análisis de algoritmos exhaustivos, con poda y Divide y Vencerás

Estudiante: Tomás Estrada Gómez

Asignatura: Algorítmica y Modelos de Computación

Lenguaje y entorno: Java 21 · IntelliJ IDEA

Memoria basada en la implementación Java facilitada y en los resultados experimentales de los archivos caso_general.csv y caso_vertical.csv.

Resumen ejecutivo

Se estudia el problema del par de puntos más cercano en el plano mediante cuatro estrategias: búsqueda exhaustiva, búsqueda exhaustiva con poda, Divide y Vencerás y Divide y Vencerás mejorado. Se ha construido una aplicación Java que permite generar y leer conjuntos de puntos en formato TSP, ejecutar las estrategias en condiciones iniciales equivalentes, visualizar la pareja mínima y comparar experimentalmente los algoritmos.

La evaluación emplea 10 muestras por cada tamaño n = 200, 500, 1500 y 5000, tanto para puntos distribuidos en dos dimensiones como para conjuntos verticales (misma coordenada X). Los resultados ponen de manifiesto la dependencia del rendimiento respecto a la distribución: en el caso general, ExhaustivoPoda es el más rápido con 5000 puntos (0,7696 ms), mientras que en el vertical destaca DyVMejorado (3,8895 ms frente a 26–29 ms de las otras estrategias).

El análisis distingue cuidadosamente el número de distancias euclídeas del tiempo total y diferencia las garantías de un algoritmo clásico de las de la implementación concreta, cuyo QuickSort ordena las franjas por Y en cada nivel recursivo.

Índice

1. Planteamiento y objetivos

2. Diseño e implementación

3. Descripción de las estrategias

4. Análisis teórico de complejidad

5. Metodología experimental

6. Resultados y discusión

7. Pruebas de corrección y limitaciones

8. Conclusiones

9. Referencias y anexos

1. Planteamiento y objetivos

1.1. Definición del problema

Dado un conjunto P = {p1, …, pn} de puntos bidimensionales, cada uno con coordenadas (x, y), se busca un par de índices distintos i y j para el cual la distancia euclídea entre pi y pj sea mínima. La distancia usada es $d(pi, pj) = \sqrt{[(xi - xj)^2 + (yi - yj)^2]}$. El problema aparece en geometría computacional, tratamiento de datos espaciales y análisis de algoritmos.

El resultado no es solamente una distancia: debe incluir los dos puntos seleccionados y un contador de distancias euclídeas evaluadas. Las restas de coordenadas utilizadas para podas no se contabilizan como distancias euclídeas, siguiendo el criterio del enunciado.

1.2. Objetivos

- Implementar cuatro estrategias con distinta organización del trabajo y comparar sus resultados.
- Garantizar conjuntos de entrada idénticos para cada estrategia mediante copias del array de puntos.
- Permitir generación aleatoria, lectura y escritura de archivos .tsp y representación gráfica de la solución.
- Realizar un estudio empírico de tiempo y número de distancias calculadas en dos distribuciones de entrada.
- Justificar las diferencias observadas mediante el análisis de operaciones elementales y recurrencias.

1.3. Alcance

El estudio se centra en encontrar el par más cercano en el plano, no en resolver el problema del viajante (TSP). Los archivos .tsp se emplean únicamente como formato de almacenamiento de puntos; los algoritmos de esta práctica siempre calculan distancias euclídeas entre las coordenadas disponibles, incluso si un fichero TSPLIB contiene metadatos de otro modelo de distancia.

2. Diseño e implementación

2.1. Organización del proyecto

Paquete	Clases relevantes	Responsabilidad
modelo	Punto, Solucion	Coordenadas, identificadores, pareja elegida, mínimo y contador.
algoritmos	Exhaustivo, ExhaustivoPoda, DivideYVenceras, DivideYVencerasMejorado, QuickSort	Resolución y ordenación por X/Y.
datos	Dataset, GeneradorDataset, ArchivoTSP	Persistencia, generación y copia de muestras.
experimentos	PruebasAlgoritmos, ExperimentosRendimiento, ControlDataset	Validación, comparaciones y exportación CSV.
vista	VentanaPrincipal, PuntosCanvas, GraficaRendimiento	Interfaz Swing, plano y gráficas.
utilidades	Distancias	Distancia euclídea.

2.2. Representación de los datos

Punto almacena id, x e y. Solucion registra los dos puntos de la mejor pareja, la distancia mínima encontrada y cuántas distancias euclídeas han sido calculadas. Dataset identifica el conjunto y ofrece una copia superficial

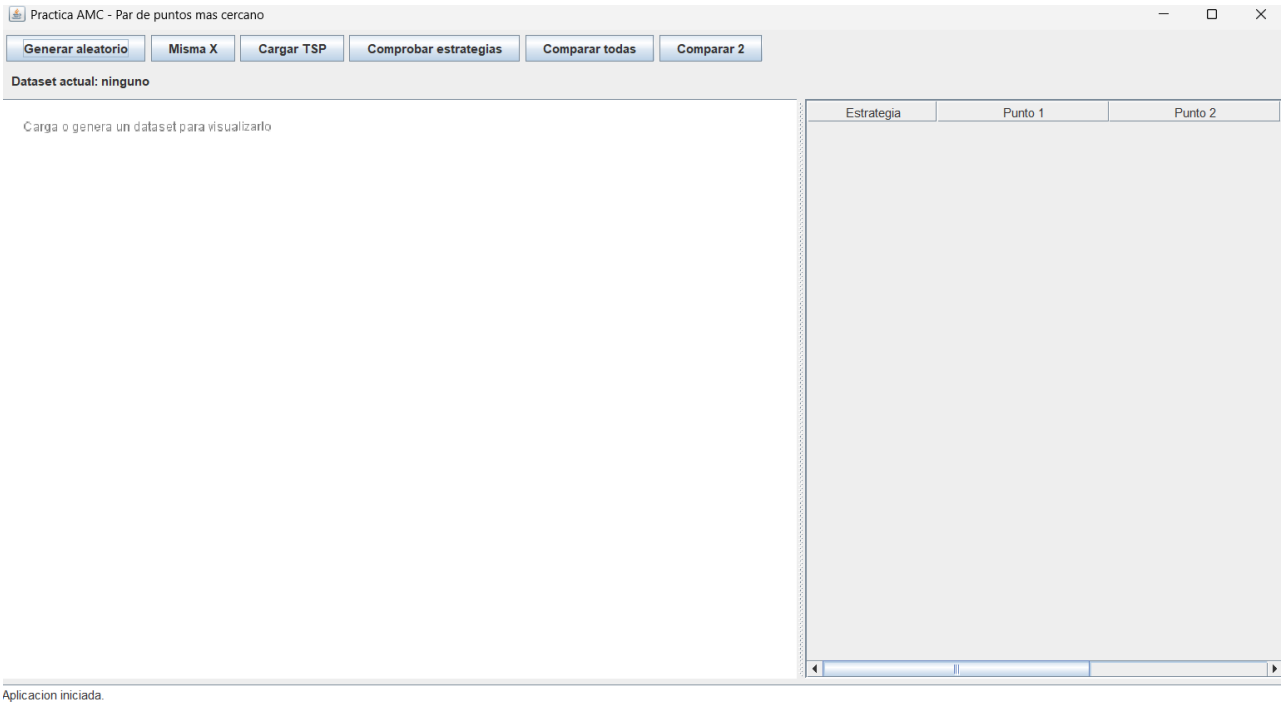
del array mediante clone(): de esta manera cada estrategia puede ordenar las referencias sin alterar el orden del dataset de partida.

Se seleccionó QuickSort para cumplir el requisito de implementar una ordenación propia. Existen dos rutinas para ordenar por X o por Y. El pivote se obtiene de la posición central del intervalo y se intercambian referencias a objetos Punto. La ordenación previa está incluida en el tiempo medido para los algoritmos que la necesitan.

2.3. Entrada/salida e interfaz

ArchivoTSP interpreta las cabeceras NAME y DIMENSION y lee la sección NODE_COORD_SECTION (id, X, Y); al escribir, usa diez cifras decimales. La aplicación dispone de interfaz Swing basada en JFrame y Canvas: permite cargar/generar conjuntos de puntos, visualizar la pareja mínima mediante un segmento y mostrar una tabla de resultados. Las comparaciones se ejecutan con SwingWorker para evitar bloquear el hilo de eventos; GraficaRendimiento muestra las curvas de tiempo y distancias.

Para comprobar las cuatro estrategias se trabaja con copias del mismo conjunto y se generan ficheros con los puntos en el orden que deja cada algoritmo, conforme al enunciado.



3. Descripción de las estrategias

3.1. Exhaustivo

Se enumeran una sola vez todas las parejas no ordenadas. El bucle exterior escoge un punto y el interior recorre los índices posteriores; para cada pareja se calcula la distancia y se actualiza Solucion si mejora el mínimo.

```
mejor ← +∞
para i ← 0 hasta n − 1:
    para j ← i + 1 hasta n − 1:
        actualizar mejor con distancia(P[i], P[j])
devolver mejor
```

Su principal ventaja es la simplicidad y la independencia respecto a la distribución espacial; su inconveniente es el crecimiento cuadrático del trabajo.

3.2. Exhaustivo con poda

Primero se ordenan los puntos por X. Para cada punto Pi se recorren los siguientes mientras Xj − Xi sea menor que la mejor distancia disponible. Al aumentar X, cuando la diferencia horizontal iguala o supera el mínimo, ningún punto más a la derecha puede mejorar el resultado.

```
ordenar P por X
mejor ← +∞
para i ← 0 hasta n − 1:
    j ← i + 1
    mientras j < n y P[j].x − P[i].x < mejor:
        actualizar mejor con distancia(P[i], P[j])
        j ← j + 1
```

La poda evita muchas distancias euclídeas cuando los puntos están repartidos horizontalmente, pero resulta ineficaz si todos tienen la misma X.

3.3. Divide y Vencerás básico

Tras ordenar por X, cada llamada divide el intervalo en dos mitades. Se resuelven recursivamente, se conserva la mejor de ambas soluciones y se analizan las parejas con un punto a cada lado de la división que aún podrían mejorar el mínimo. El código aplica podas por diferencias horizontales y acumula los contadores recursivos sin volver a calcular las distancias ya almacenadas.

```
resolver(inicio, fin):
    si hay 1 o 2 puntos: resolver directamente
    medio ← (inicio + fin) / 2
    izquierda ← resolver(inicio, medio)
    derecha ← resolver(medio+1, fin)
    mejor ← min(izquierda, derecha)
    examinar cruces de la zona central con poda por X
    devolver mejor
```

La combinación es más barata en distribuciones uniformes que en conjuntos verticales, donde la poda por X no reduce el número de candidatos.

3.4. Divide y Vencerás mejorado

La versión mejorada mantiene la división recursiva y restringe la búsqueda central a una franja alrededor de la división de anchura relacionada con la distancia mínima. Los candidatos se recopilan en un ArrayList, se convierten a array y se ordenan por Y con el QuickSort propio.

Para cada candidato se examinan como máximo las once posiciones siguientes en el orden Y; se detiene la búsqueda si la diferencia vertical alcanza la distancia mínima. Además se registra, en un IdentityHashMap, la pertenencia de cada objeto Punto a la mitad izquierda o derecha. Solo se calcula la distancia entre candidatos de mitades distintas, evitando repetir comparaciones resueltas recursivamente.

```
resolver(inicio, fin):
    resolver las dos mitades y conservar el mejor mínimo
    franja ← puntos con |x − x_medio| < mínimo
    registrar mitad original con IdentityHashMap
    ordenar franja por Y
    para cada candidato i:
        examinar hasta 11 candidatos posteriores j
```

```
        detener si Yj − Yi ≥ mínimo
        si mitades(i) ≠ mitades(j): calcular distancia
    devolver mejor
```

La optimización reduce el contador, pero tiene un coste adicional en creación de arrays, ordenaciones y consultas al mapa. Por ello resulta incorrecto deducir automáticamente un menor tiempo a partir de un menor número de distancias.

4. Análisis teórico de complejidad

4.1. Criterio y notación

Sea n el número de puntos. Se distingue C(n), número de evaluaciones efectivas de la distancia euclídea, y T(n), tiempo total, incluyendo comparación de coordenadas, ordenación, construcción de listas y mapas y llamadas recursivas. Las cotas describen la implementación facilitada, no una variante idealizada. En particular, QuickSort tiene coste medio O(n log n) y peor caso $O(n^2)$; su pivote central por posición no garantiza particiones equilibradas en todos los órdenes de entrada.

4.2. Exhaustivo

Para cada i se consideran n − i − 1 candidatos. Por tanto, C(n) = (n−1) + (n−2) + … + 1 = n(n−1)/2. En consecuencia, $T(n) = \Theta(n^2)$ en cualquier distribución espacial. Mejor, medio y peor caso coinciden en orden asintótico y en número exacto de distancias.

4.3. Exhaustivo con poda

La ordenación inicial cuesta O(n log n) en promedio. Después, si Ki representa el número de sucesores que superan la prueba de poda para el punto i, C(n) = Σ Ki, con $0 \le C(n) \le n(n-1)/2$. Cuando el mínimo se reduce pronto y las separaciones horizontales descartan candidatos, el barrido posterior puede ser cercano a lineal y el tiempo queda dominado por la ordenación. En la entrada vertical todas las diferencias de X son cero y, mientras la distancia mínima sea positiva, se examinan todas las parejas: C(n) = n(n−1)/2. El peor tiempo es $O(n^2)$; en promedio no se impone una cota más precisa sin un modelo probabilístico de distribución.

4.4. Divide y Vencerás básico

En cada nivel, la parte recursiva responde a 2T(n/2). Examinar cruces de la franja cuesta O(n) si el número de candidatos relevantes permanece controlado, lo que da T(n) = 2T(n/2) + O(n) y O(n log n) de trabajo recursivo (más ordenación inicial). Este escenario favorable es consistente con la hipótesis de distribución uniforme del enunciado. Sin embargo, en la implementación la zona central puede contener Θ(n) puntos en cada mitad; examinar cruces admite $\Theta(n^2)$ operaciones en una llamada. Así, $T(n) = 2T(n/2) + O(n^2)$ produce $O(n^2)$ en el peor caso. No debe presentarse O(n log n) como una garantía para cualquier distribución.

4.5. Divide y Vencerás mejorado

La selección de la franja y el recorrido de hasta 11 vecinos tienen coste lineal respecto al tamaño de la subinstancia. La ordenación por Y se repite en cada llamada y, con QuickSort en comportamiento medio, cuesta O(k log k) para una franja de $k \le n$ puntos. Una cota general media para la combinación es O(n log n), por lo que la recurrencia $T(n) = 2T(n/2) + O(n \log n)$ conduce a $O(n \log^2 n)$. La ordenación inicial por X no modifica ese orden dominante. En un escenario adverso de QuickSort, una ordenación de franja puede costar $O(n^2)$ y la cota conservadora del procedimiento completo es $O(n^2)$.

La versión clásica óptima del par más cercano alcanza O(n log n) manteniendo estructuras ordenadas por Y entre niveles; no se debe atribuir automáticamente ese resultado a esta implementación, que reordena la franja en cada llamada. Al contrario, el número de distancias euclídeas de la versión mejorada queda acotado

por O(n log n) al evaluar como máximo 11 candidatos por punto en cada nivel, con independencia del coste de ordenar.

4.6. Comparación de órdenes de crecimiento

Estrategia	Situación favorable	Comportamiento medio / hipótesis	Peor caso
Exhaustivo	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
ExhaustivoPoda	O(n log n), si el barrido es lineal	Depende de la distribución	$O(n^2)$
DivideVenceras	O(n log n), cruces controlados	O(n log n) bajo hipótesis de franjas pequeñas	$O(n^2)$
DyVMejorado	$O(n \log^2 n)$ con ordenación media	$O(n \log^2 n)$ con ordenación media	$O(n^2)$ por QuickSort adverso

Estas cotas de tiempo no deben confundirse con el número de distancias: por ejemplo, DyVMejorado puede calcular muy pocas distancias pero tardar más que ExhaustivoPoda en el caso general por sus costes auxiliares.

5. Metodología experimental

5.1. Diseño del ensayo

Se utilizaron cuatro tallas: 200, 500, 1500 y 5000 puntos. Para cada talla y modalidad se generaron diez conjuntos distintos; las cuatro estrategias recibieron copias de cada mismo conjunto. La operación cronometrada abarca la ejecución del método resolver(), incluida la ordenación requerida, pero excluye la generación del conjunto y la escritura del CSV. Los tiempos se tomaron con System.nanoTime() y se convirtieron a milisegundos. Antes de medir, se realizaron tres pasadas de calentamiento sobre un conjunto de 200 puntos para las estrategias seleccionadas.

Los datos provienen de caso_general.csv y caso_vertical.csv obtenidos durante el desarrollo. Los CSV guardan las medias por tamaño y algoritmo, no las diez observaciones individuales. Por tanto, no se calculan desviaciones típicas ni intervalos de confianza; pequeñas diferencias entre algoritmos muy rápidos deben interpretarse con cautela.

5.2. Tipos de entrada

Caso general. Se generan X e Y de forma independiente con Random.nextDouble() sobre el intervalo [−1000, 1000). Se obtiene una distribución espacial bidimensional aproximadamente uniforme.

Caso vertical. Todos los puntos tienen X = 5. La coordenada Y se calcula como 0,73·i + 0,5·u, con u aleatorio en [0,1). De este modo las Y crecen estrictamente y los puntos no coinciden entre sí. Es una entrada especial que anula la poda basada en X; no coincide exactamente con el generador de ejemplo en C++ del enunciado.

5.3. Entorno y reproducibilidad

Los experimentos se realizaron utilizando Java 21 y el entorno de desarrollo IntelliJ IDEA, sobre un equipo con sistema operativo Windows 11, procesador Ryzen 7 y 16 GB de memoria RAM. Los generadores no utilizan semilla fija, por decisión de implementación: las comparaciones dentro de una ejecución son justas, pero no se reproducen exactamente los mismos puntos en ejecuciones independientes.

La métrica C(n) cuenta únicamente distancias euclídeas efectivamente evaluadas. La comparación por coordenadas X o Y no incrementa el contador. Se comprueba que cada estrategia devuelve la misma distancia mínima dentro de una tolerancia numérica de 10^{-8}.

6. Resultados y discusión

6.1. Resultados del caso general

Tabla 1. Tiempo medio de ejecución (ms), caso general.

n	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyVMejorado
200	0.2291	0.0801	0.0312	0.1344
500	0.5002	0.1800	0.0833	0.3102
1500	2.5097	0.6424	0.4327	1.5512
5000	27.2608	0.7696	3.4850	5.2057

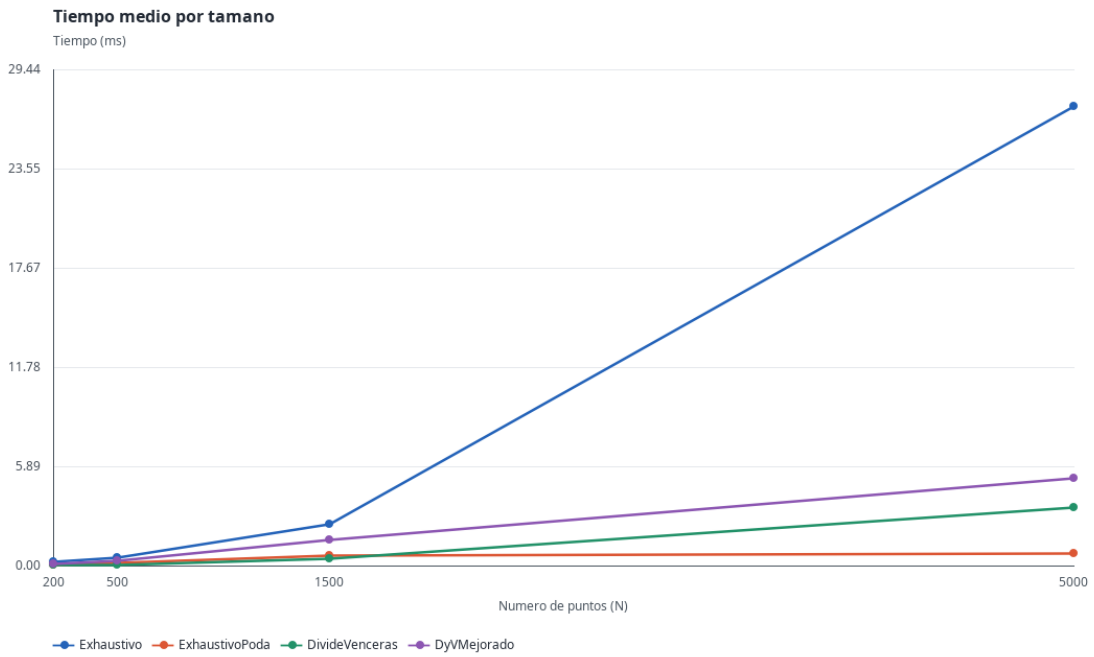


Figura 1. Tiempo medio según tamaño: caso general.

Para n = 5000, ExhaustivoPoda obtuvo 0,7696 ms, DivideVenceras 3,4850 ms, DyVMejorado 5,2057 ms y Exhaustivo 27,2608 ms. El algoritmo con poda fue el más rápido en ese tamaño, mientras que la versión mejorada sufrió el sobrecoste de reconstruir y ordenar franjas. En n = 200 y n = 500, DivideVenceras aparece más rápido que ExhaustivoPoda; dado lo breves que son estas mediciones, las diferencias pequeñas no deben generalizarse sin más repeticiones y observaciones individuales.

Tabla 2. Distancias euclídeas medias, caso general.

n	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyVMejorado
200	19 900.0	318.9	680.4	171.4
500	124 750.0	652.9	2 123.3	438.9
1500	1 124 250.0	2 296.4	9 390.3	1 275.2
5000	12 497 500.0	6 093.9	47 860.4	4 338.9

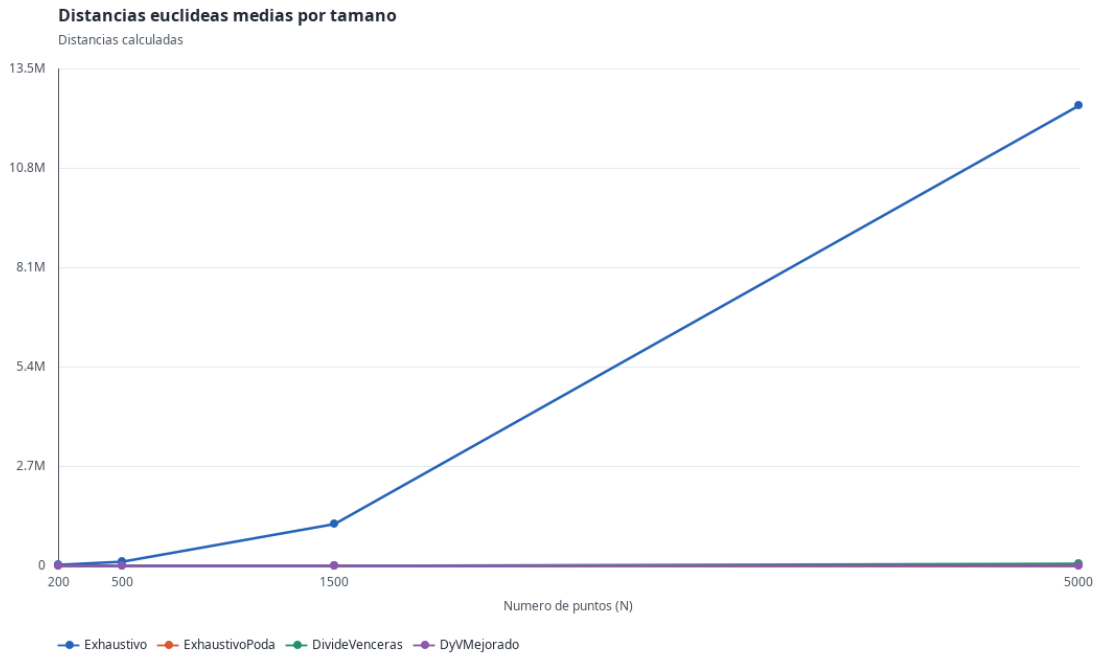


Figura 2. Distancias calculadas según tamaño: caso general.

La comparación de contadores cambia la clasificación: con 5000 puntos, DyVMejorado calcula 4338,9 distancias de media, frente a 6093,9 de ExhaustivoPoda, 47860,4 de DivideVenceras y 12 497 500 de Exhaustivo. En este caso DyVMejorado realiza aproximadamente un 28,8 % menos de distancias que ExhaustivoPoda, pero tarda más. Ello confirma que minimizar las operaciones medidas no equivale a minimizar el tiempo total.

6.2. Resultados del caso vertical

Tabla 3. Tiempo medio de ejecución (ms), caso vertical.

n	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyVMejorado
200	0.0452	0.1963	0.0571	0.1093
500	0.2708	0.3236	0.3344	0.2820
1500	2.4317	2.6164	2.7136	1.1528
5000	26.5856	27.0663	28.7716	3.8895

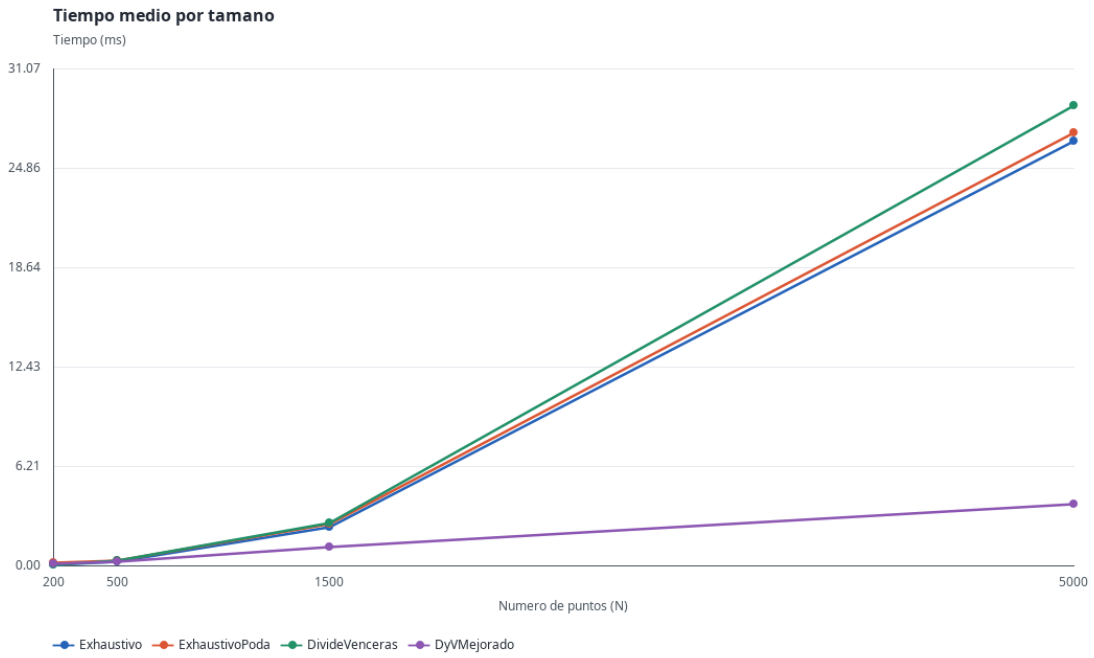


Figura 3. Tiempo medio según tamaño: caso vertical.

Con n = 5000, DyVMejorado tarda 3,8895 ms; Exhaustivo, 26,5856 ms; ExhaustivoPoda, 27,0663 ms; y DivideVenceras, 28,7716 ms. El mejorado es aproximadamente 6,8 veces más rápido que el exhaustivo en esta muestra. Esta ventaja no tiene por qué apreciarse igual en n = 200, donde los gastos auxiliares y el ruido de medición pesan mucho más.

Tabla 4. Distancias euclídeas medias, caso vertical.

n	Exhaustivo	ExhaustivoPoda	DivideVenceras	DyVMejorado
200	19 900.0	19 900.0	19 900.0	106.4
500	124 750.0	124 750.0	124 750.0	339.1
1500	1 124 250.0	1 124 250.0	1 124 250.0	966.5
5000	12 497 500.0	12 497 500.0	12 497 500.0	3 405.8

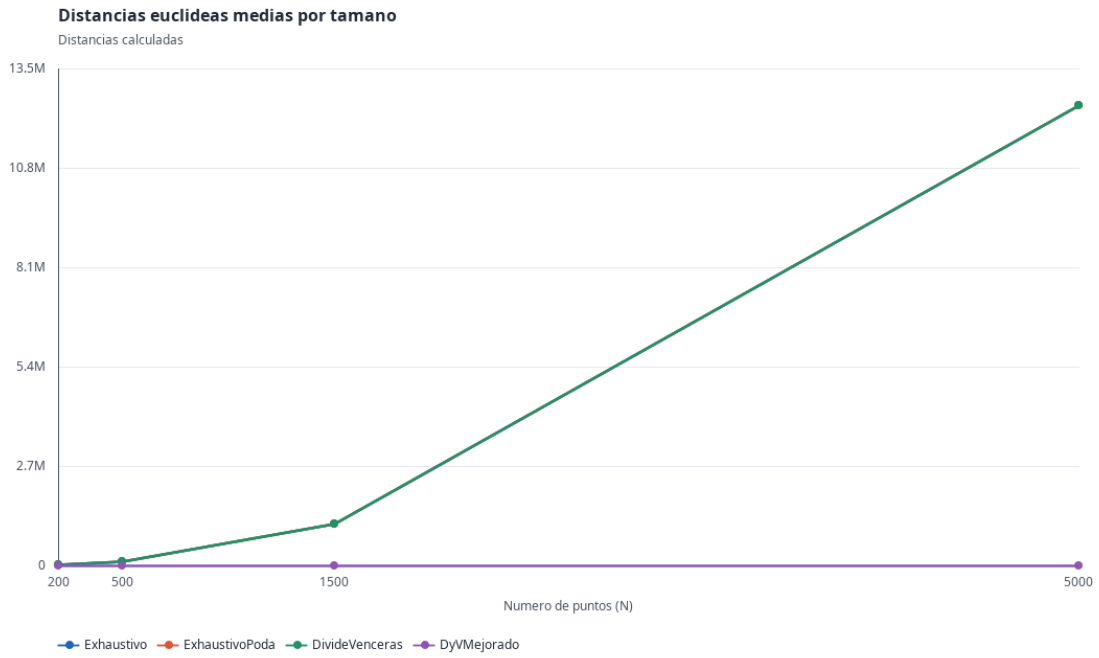


Figura 4. Distancias calculadas según tamaño: caso vertical.

En el caso vertical, Exhaustivo, ExhaustivoPoda y DivideVenceras realizan el mismo número de distancias: 19 900 (n = 200), 124 750 (n = 500), 1 124 250 (n = 1500) y 12 497 500 (n = 5000). La poda horizontal no elimina parejas porque todas las diferencias de X son cero, y la división básica sitúa todos los puntos en las franjas pertinentes. Por el contrario, DyVMejorado calcula solo 3405,8 distancias de media con 5000 puntos, una reducción del 99,97 % respecto al exhaustivo.

6.3. Interpretación comparada

Los dos escenarios muestran por qué no basta con estudiar solamente el orden de complejidad o solamente el contador de distancias. Para datos generales, la poda por X suele descartar candidatos a bajo coste, de modo que ExhaustivoPoda se beneficia de una implementación simple. En la entrada vertical esa ventaja desaparece y la ordenación por Y del mejorado se vuelve decisiva. Las cifras del caso vertical son particularmente útiles para ilustrar el peor caso de la poda horizontal, aunque el patrón de Y está diseñado con separaciones crecientes y no representa todas las posibles distribuciones verticales.

El exhaustivo ofrece un control experimental: el valor esperado $C(n) = n(n-1)/2$ coincide exactamente con los cuatro tamaños de las tablas. Esta coincidencia ayuda a verificar que el contador se incrementa una sola vez por pareja.

7. Pruebas de corrección y limitaciones

7.1. Validación funcional

Antes del estudio definitivo se ejecutaron 69 pruebas automáticas: nueve casos especiales, 50 conjuntos aleatorios (diez por cada uno de cinco tamaños), cinco conjuntos con coordenadas enteras potencialmente repetidas y cinco conjuntos verticales. Las 69 pruebas se completaron sin discrepancias entre distancias mínimas de las cuatro estrategias, según el resumen de ejecución compartido. También se verificó una prueba de escritura y lectura de 100 puntos TSP, comprobando la dimensión y los identificadores y coordenadas con tolerancia.

Estas pruebas aportan evidencia empírica de funcionamiento, pero no sustituyen a una demostración matemática general. En particular, los casos con menos de dos puntos están fuera de las entradas válidas admitidas por Dataset y GeneradorDataset.

7.2. Limitaciones y mejoras posibles

- Reproducibilidad: almacenar semilla o guardar las diez muestras individuales permitiría repetir exactamente cada experimento.
- Benchmark: registrar tiempos individuales, realizar más calentamiento y usar un banco de pruebas específico como JMH mejoraría la precisión temporal.
- Algoritmo mejorado: mantener un orden por Y propagado entre recursiones podría reducir el tiempo a O(n log n) en su versión clásica.
- Generación de entrada: revisar la equivalencia con el generador del ejemplo en C++ del enunciado si se exige reproducir sus distribuciones.
- Comparación visual: añadir escalas logarítmicas o gráficas ampliadas facilitaría observar las estrategias rápidas cuando el exhaustivo domina el eje vertical.

8. Conclusiones

La práctica ha permitido comparar cuatro formas de abordar un mismo problema geométrico. La estrategia exhaustiva ofrece una solución sencilla, con un contador exactamente cuadrático. La poda horizontal reduce notablemente el trabajo en distribuciones generales y resulta muy rápida cuando el mínimo hallado descarta numerosos candidatos. Divide y Vencerás básico introduce recursividad, pero su comportamiento depende de la densidad de puntos de la zona central; en el escenario vertical puede perder la ventaja de la partición.

Divide y Vencerás mejorado evita esa degradación en el número de distancias mediante la ordenación por Y, la limitación a once vecinos posteriores y la exclusión de parejas pertenecientes a la misma mitad. No obstante, en nuestra implementación debe considerarse el coste de ordenar la franja en cada nivel recursivo y de gestionar estructuras auxiliares, por lo que no siempre es el más rápido en el caso general.

Los experimentos corroboran la relevancia de estudiar el problema con entradas de distinta distribución: para n = 5000, ExhaustivoPoda logra el menor tiempo en el caso general, mientras que DyVMejorado domina el caso vertical. La evaluación conjunta de tiempo y distancias ofrece una explicación más completa que cualquiera de estas métricas por separado.

9. Referencias y anexos

[1] Enunciado de Práctica 1. «Análisis de Algoritmos Exhaustivos, con Poda y algoritmos Divide y Vencerás», versión 24.10, asignatura Algorítmica y Modelos de Computación, pp. 1–14.

[2] «Nota aclaratoria respecto al número de cálculos de distancias en la práctica 1 de AMC», documento docente facilitado con el proyecto.

[3] Código fuente Java del proyecto facilitado: paquetes algoritmos, datos, experimentos, modelo, utilidades y vista.

[4] Resultados empíricos aportados: caso_general.csv y caso_vertical.csv. Gráficas generadas mediante GraficaRendimiento.java.

Anexo A. Archivos y validaciones

Los ficheros CSV incorporan una fila por tamaño y estrategia con columnas N, Algoritmo, TiempoMedioMs y DistanciasMedias. Cada punto mostrado en la representación gráfica procede de sus coordenadas

transformadas a píxeles mediante una escala uniforme, y la solución se destaca mediante dos círculos y un segmento. La clase de pruebas automáticas y el control de archivos quedan disponibles en el proyecto para comprobar futuras modificaciones.